

GABOCHIE MARKETING

Architecture & Security Assessment

Production Readiness, Security Audit & Migration Roadmap

Ghana's #1 Local Visibility Agency

Prepared: May 14, 2026

1. Executive Summary

Production readiness assessment of Gabochie Marketing's technical stack

Gabochie Marketing's current architecture is functional for early-stage operation but has significant security and scalability issues that must be addressed before the business can grow past approximately 50-100 clients. The stack combines a production-grade frontend host (Netlify) with an MVP-grade backend (Google Sheets + Google Apps Script) and has several critical vulnerabilities including plain-text password storage, exposed credentials in source code, and no database access controls.

This report assesses the current architecture, identifies risks ranked by severity, compares the current stack against a proposed Supabase + Vercel alternative, and provides a phased 6-step migration roadmap. The first two phases (auth fix and database migration) can be completed within 2 weeks at zero additional cost and eliminate the most critical risks.

Verdict: MVP-Grade Architecture. Not Production-Ready.

The frontend (Netlify) is fine. The backend (Google Sheets + Apps Script) and auth system are the critical risks.

2. Current Architecture Overview

The application is a static HTML/CSS/JS site hosted on Netlify with a serverless backend powered by Google Apps Script. A single Google Sheet serves as the database for leads, clients, and CRM data. An admin panel (also hosted on Netlify) communicates with a separate Google Apps Script backend. The client portal uses the same sheet for authentication and dashboard data.

Layer	Technology	Assessment
Frontend Host	Netlify	Production-grade. Static hosting with CDN, SSL, security headers.
CSS Framework	Tailwind v3 (CDN)	Functional but no build step. Full library sent to every visitor.
Backend	Google Apps Script	MVP-grade. 6-min limit, 50 concurrent writes, no rate limiting.
Database	Google Sheets	MVP-grade. Flat rows, no relations, no access control.
Auth	Custom DIY	Critical risk. Plain text passwords, hardcoded creds, no hashing.
Lead Capture	Forms + localStorage	Functional. Ghost-free mode works, but no CAPTCHA or validation.

3. Security & Scalability Issues

Ranked by severity. Critical and High items should be addressed immediately.

Issue	Sev.	Detail
Google Sheets as database	Critical	10M cell limit, ~50 concurrent writes, no indexes, no relationships, no transactions. Two backends write to same sheet with
Passwords in plain text	Critical	Admin credentials (admin/gabochie2024) hardcoded in Code.gs. Client passwords stored as raw text in Clients sheet.
No encryption at rest	Critical	Google Sheets has no column-level encryption. Anyone with sheet access or who finds the Sheet ID in JS source can read
Admin creds in source	Critical	Username and password visible to anyone with Apps Script access or who finds the file in version control.
Sheet ID exposed	High	SHEET_ID hardcoded in frontend JS. Anyone can inspect and attempt to access the sheet.
No rate limiting	High	Forms can be spammed with unlimited submissions. No CAPTCHA, no throttling.
No input validation	High	No sanitization on form fields before sending to backend or storing in localStorage.
Tailwind via CDN	Medium	Full library (hundreds of KB unused CSS) sent to every visitor. No purge optimization.
GAS execution limits	Medium	6-minute cap, 20K URL fetch/day, 100 emails/day. Bottleneck at ~50+ submissions/day.
No staging env	Medium	Changes go straight to production. No preview deployments.
DIY auth fragile	Medium	24-hour session token in script properties. No refresh tokens, OAuth, or password hashing.

Summary: 4 Critical risks (passwords, encryption, Sheets-as-DB, exposed credentials) and 3 High risks need immediate attention. The Medium risks are important but not urgent. The Critical items alone represent genuine liability exposure for a business handling client contact information.

8. Google Sheets vs. Supabase (PostgreSQL)

Why migration to a proper database is recommended

Feature	Google Sheets (Current)	Supabase (PostgreSQL)
Data model	! Flat rows, no relationships	+ Full relational (clients, invoices, referrals)
Security	! No access control, plain text	+ Row-Level Security, encrypted at rest, proper auth
Concurrent writes	! ~50 max	+ Thousands
Scaling	! Hits limits at ~10K rows	+ Millions of rows
Backups	! Manual only	+ Automated daily snapshots
Auth	! DIY, broken, plain text	+ Built-in: Google OAuth, email/password, magic link
APIs	! Custom Apps Script	+ Auto-generated REST + GraphQL
Real-time	! No	+ Built-in WebSockets
Cost	Free (for now)	Free tier (500 MB, 50K users) -> \$25/mo production
Rate limiting	! None	+ Built-in row-level security policies
Input validation	! None	+ Database constraints + RLS policies

Supabase offers a generous free tier that is sufficient for Gabochie Marketing's current scale (500 MB database, 50K users, 2 GB bandwidth). The paid tier (\$25/mo) unlocks 8 GB database, 100K users, and 50 GB bandwidth - more than enough for the Year 1-2 scenario. The migration from Google Sheets to Supabase eliminates every Critical and High risk identified in this report.

5. Migration Roadmap

6 phases from current state to production-ready architecture

PHASE 1: Fix Authentication

Week 1 -- Highest priority

#	Action	Effort	Rationale
1	Hash passwords using bcrypt	1 day	Eliminates plain-text exposure. No user data at risk.
2	Implement Supabase Auth with magic link or Google OAuth	2 days	Replaces DIY auth with battle-tested system.
3	Remove hardcoded admin credentials from source code	1 day	Admin login via Supabase Auth instead of plain-text check.

PHASE 2: Database Migration

Week 1-2 -- Core data layer

#	Action	Effort	Rationale
4	Create Supabase project, set up schema (leads, clients, referrals, invoices)	2 days	Relational model with foreign keys, indexes, constraints.
5	Write migration script to export Google Sheets data to Supabase	1 day	One-time export preserving all existing client and lead records.
6	Build REST API endpoints with RLS policies for each table	2 days	Replace Apps Script endpoints with secure Supabase API.

PHASE 3: Vercel Serverless Functions

Week 2-3 -- Replace Google Apps Script

#	Action	Effort	Rationale
7	Deploy form submission handler as Vercel Edge Function	1 day	Replaces POST handler in google-apps-script-backend.js.
8	Deploy admin CRM API as serverless functions	2 days	Replaces all admin/Code.gs functionality.
9	Deploy client portal auth and dashboard endpoints	1 day	Client login and dashboard data via secure Supabase queries.

PHASE 4: Security Hardening

Week 3 -- Rate limiting, validation, CSP

#	Action	Effort	Rationale
10	Add CAPTCHA to all forms (Turnstile or reCAPTCHA v3)	1 day	Prevents spam submissions and bot attacks.
11	Add server-side input validation and sanitization	1 day	Prevents XSS, SQL injection, and malformed data.
12	Add rate limiting on all API endpoints	1 day	Prevents abuse. 10 req/min per IP on forms.

PHASE 5: Frontend Optimization

Week 3-4 -- Build step, staging

#	Action	Effort	Rationale
13	Add Vite build step: auto-purge Tailwind unused CSS, minify HTML/JS	2 days	Reduces page weight from ~800KB to ~150KB.
14	Set up Vercel preview deployments per branch	1 day	Every PR gets a unique URL for testing before production.
15	Remove SHEET_ID and sensitive strings from client-side JS	1 day	No backend identifiers exposed to visitors.

PHASE 6: Vercel Migration (Optional)

Week 4+ -- Move off Netlify if desired

#	Action	Effort	Rationale
16	Migrate static hosting from Netlify to Vercel	1 day	Consolidates hosting + serverless on one platform.
17	Set up custom domain, SSL, redirects on Vercel	1 day	Same domain, zero downtime migration.
18	Remove Google Sheets and Apps Script entirely	1 day	Complete decoupling from legacy backend.

Recommendation

Do Phase 1 + 2 next week. The rest can follow incrementally.

Fix auth and passwords FIRST - this is an active liability.

Migrate data from Google Sheets to Supabase - zero cost, fixes worst issues.

Replace Google Apps Script with Vercel serverless functions.

Add rate limiting, CAPTCHA, and input validation.

Optimize frontend with build step and preview deploys.

Move to Vercel only if you want consolidation.

marketing.gabochie.com

gabochiemarketing@gmail.com

+233 50 638 4917